

4. RESULTADOS E DISCUSSÃO

4.1 Avaliação de Desempenho e Comportamento da Camada de Cache

Para mensurar a viabilidade técnica e a eficiência operacional do microsserviço proposto, realizou-se uma rotina controlada de testes empíricos automatizados via cliente assíncrono HTTP (httpx). O experimento avaliou duas situações centrais de operação no ciclo de vida da API:

- 1. **Requisição a Frio (Cache Miss):** Cenário em que o payload do produto é submetido pela primeira vez. A aplicação realiza o parsing do contrato via Pydantic, dispara a requisição de inferência estruturada para a API remota do Gemini com imposição de JSON Schema e persiste o resultado em memória.
- 2. **Requisição com Cache Ativo (Cache Hit):** Cenário subsequente em que o mesmo identificador de produto (SKU) e atributos são submetidos novamente, sendo interceptados pelo hash determinístico SHA-256 e resolvidos diretamente em memória volátil (RAM), sem acionamento do provedor de inteligência artificial.

Os dados quantitativos observados no ensaio estão consolidados na Tabela 1.

Tabela 1 – Comparativo de métricas operacionais entre chamadas a frio e chamadas em cache

Parâmetro de Avaliação	Requisição Inicial (Cache Miss)	Requisição Repetida (Cache Hit)	Variação Observada
Tempo de Resposta Total (Cliente)	13.780,90 ms (~13,78 s)	1,64 ms	-99,98%
Tempo de Processamento Interno	13.778,50 ms	0,82 ms	-99,99%
Custo de Inferência (Tokens LLM)	Consumo integral de cota	Zero tokens consumidos	-100,00%
Integridade de Esquema (JSON)	Validado via Pydantic	Validado via Pydantic	Invariante

Código de Estado HTTP	200 OK	200 OK	—
-----------------------	--------	--------	---

Fonte: Dados da pesquisa (2026).

4.2 Análise dos Dados e Discussão Técnica

Os resultados demonstram que a latência de ponta a ponta na primeira chamada (13,78 segundos) é dominada pelo tempo de resposta da rede externa e pelo processo de inferência estocástica guiada por restrições formais de saída (*Structured Outputs*). Embora esse valor seja aceitável para fluxos assíncronos ou rotinas de catálogo em lote (*batch processing*), torna-se um gargalo se aplicado repetidamente em operações de consulta interativa em ambientes de retalho.

Com a aplicação do mecanismo de *caching* determinístico baseado em hash, o tempo de entrega caiu para **1,64 ms**, registrando uma diminuição superior a **99,9%** no tempo de resposta do cliente.

Do ponto de vista arquitetural, esse comportamento confirma duas hipóteses fundamentais do projeto:

- **Mitigação de custos e concorrência:** Produtos estáveis ou já processados no catálogo não degradam o limite de requisições por minuto (RPM/TPM) do provedor de LLM, blindando o microserviço contra erros de saturação (como HTTP 429 ou 503).
- **Determinismo de contrato:** O cliente HTTP recebe rigorosamente a mesma estrutura tipada de dados para os três canais (Instagram, WhatsApp e E-commerce), eliminando o risco de regressão sintática em requisições recorrentes.